

ENEL712 — Exam Question Guide

Registers · Formulas · Code Templates · Worked Examples

Companion to the Study Guide. Study Guide = *what/why*. This document = *how/calculate*. Use both together.

Universal Exam Technique — apply to every question

(1) Identify physical quantities to sense/control. (2) Trace data flow: *sensor* → *MCU peripheral* → *software* → *output/comms*. (3) Justify each component with a constraint. (4) Calculate: ADC sample rate, buffer size, OCR, UART baud, battery life. (5) State the binding *constraint*: SRAM 8 KB limit, EEPROM endurance, wireless TX current, baud error <2%.

Checklist: ☐ physical quantities ☐ data flow ☐ sample rate ☐ buffer/SRAM check ☐ protocol + wire count ☐ OCR value ☐ interrupt vs poll justification ☐ battery life I_{avg} ☐ FSM states ☐ modules + source files ☐ MCU vs MPU justification

Contents

1 The Two Exam Scenarios	2
2 AT90USB1287 — Specifications & Memory Map	2
3 Architecture Decisions	3
3.1 MCU vs MPU	3
3.2 ARM Cortex-M comparison	3
3.3 Clock accuracy	3
3.4 2-stage pipeline and flush	3
4 IoT Hierarchy & Hardware Design	3
4.1 IoT architecture layers	3
4.2 Hardware design process — always I/O first	3
4.3 I/O device categories	4
4.4 Power supply: LDO vs buck	4
5 Software Engineering	4
5.1 4 process activities	4
5.2 Functional vs non-functional requirements	4
5.3 Hardware vs software partitioning	4
5.4 FSM code skeleton	4
6 All Key Formulas — Reference Table	5
7 Register Configuration & Code Templates	7
7.1 Timer1 CTC at 200 Hz – ADC trigger	7
7.2 ADC init + result ISR	7
7.3 Timer1 Fast PWM (OC1A)	7
7.4 Input Capture (slot timestamps)	8
7.5 PID controller (fixed-point integer)	8
7.6 PROGMEM + EEPROM access	8
7.7 Watchdog Timer — enable and reset	8
7.8 SREG save/restore inside an ISR	8
7.9 I ² C read transaction skeleton	8
7.10 I ² C / TWI initialisation	9
7.11 SPI initialisation (master)	9
7.12 Sleep mode entry (Vineyard power-down)	9
7.13 Event-driven main loop template (disk project)	9

8	Communication Protocol Selection	10
9	Software Architecture & FSM Diagrams	10
9.1	Modular design (3-level hierarchy)	10
10	Worked Examples — Q1 to Q5	11
11	Extended Scenarios — Full System Design	11
12	Practice Questions — Q1 to Q15	14
13	Additional Scenario — USB Audio / Data Logger	15
	Quick Reference — All Key Formulas	16
14	Abbreviations & Unit Relationships	17
14.1	Abbreviations	17
14.2	Unit Relationships	19

The Two Exam Scenarios

Scenario 1 — Disk Synchronisation

Two slotted disks (perpendicular axes) must pass a shared zone without collision. Tachogenerator → ADC speed; PWM → motor; Input Capture → slot timestamps; UART/USB → PC data; PID → speed control.

Scenario 2 — Vineyard IoT Weather Station

Remote environmental monitoring (temp, humidity, rainfall, sunlight). Battery powered. ZigBee wireless. I²C sensors (HTS221, DS3231). Aggressive sleep (< 1 μA). EEPROM for ZigBee channel and calibration.

Component	Disk Sync	Vineyard IoT
ADC	Tachogenerator voltage → RPM	Analog light/rainfall sensors
PWM	Motor speed control (OC1A)	Not primary
Timer CTC	ADC trigger at 200 Hz	WDT wakeup every 60 s
Input Capture	Slot event timestamps	Not used
UART/USB	Stream 30-s data log to PC	ZigBee (XBee UART)
I ² C	Not primary	Temp 0x48, humidity 0x5F, RTC 0x68
Sleep modes	Not priority	Critical — months of battery
EEPROM	PID gains (K_p, K_i, K_d)	ZigBee channel, calibration
SRAM	Circular buffer (8 KB hard limit!)	Small sensor buffers
State machine	IDLE/RAMP/STEADY/STOP	SLEEP/SENSE/TX/SLEEP

AT90USB1287 — Specifications & Memory Map

Parameter	Value / Design note
Architecture	8-bit AVR RISC, Harvard
Clock	Up to 16 MHz; <i>crystal required</i> for comms
Flash	128 KB; PROGMEM for const tables
SRAM	8 KB (0x0100–0x20FF)
EEPROM	4 KB, 100k cycles, 3.4 ms/write
ADC	10-bit, 8 ch, max ≈9.6 kHz sample rate
Timers	T0/T2 (8-bit), T1/T3 (16-bit)
Interfaces	SPI, UART, TWI (I ² C), USB FS
Sleep (Power-down)	< 1 μA; wake: ext INT, WDT
Active current	≈10–20 mA at 16 MHz

Address	Region	Used for
0x0000–0x001F	Registers R0–R31	CPU general-purpose
0x0020–0x00FF	I/O registers	Peripheral control
0x0100–0x20FF	SRAM 8 KB	Globals up; stack down

Pitfall

Stack grows **down** from 0x20FF; globals grow **up** from 0x0100. Large local arrays or deep ISR call chains collide silently.

Sleep mode	Current / Wake source
Idle	$\approx 5\text{ mA}$ / any interrupt
ADC Noise Red.	$\approx 1\text{ mA}$ / ADC complete
Power-down	$< 1\text{ }\mu\text{A}$ / ext INT, WDT, TWI
Power-save	$< 1\text{ }\mu\text{A}$ / Timer2 async, ext INT

Architecture Decisions

MCU vs MPU

Requirement	Choice	Why
Run full Linux OS	MPU	MCU has no MMU
Complex graphical touchscreen UI	MPU	Needs GPU, GB RAM
Gigabit Ethernet / USB host	MPU	Hardware too complex for 8-bit
More than 2 MB RAM	MPU	MCU SRAM is KB-range
$< 1\text{ }\mu\text{s}$ interrupt response	MCU	OS scheduling jitter rules out MPU
Battery powered, years of operation	MCU	MPU power too high ($> 500\text{ mA}$)
Motor control, sensors, simple comms	MCU	Sufficient, cheaper, simpler

ARM Cortex-M comparison

Core	DMIPS	Key feature	Choose for
Cortex-M0+	52	Lowest power	Simple IoT, replacing 8-bit MCUs
Cortex-M3	125	Hardware divide	General embedded control
Cortex-M4	180	DSP + FPU	Motor drives, audio, signal processing
Cortex-M7	300+	Dual-issue	High-performance real-time
Cortex-A5	850	MMU — runs Linux	IoT gateway, Raspberry Pi-class

Choose ARM over AT90USB when

(1) Floating-point required (PID with float, FFT). (2) Need $> 128\text{ KB}$ Flash or $> 8\text{ KB}$ RAM. (3) Need faster comms (USB full-speed 12 Mbps, Ethernet). (4) Control loop too tight for 16 MHz. Do not upgrade simply because it is more powerful — ARM needs more complex init, higher power, and higher cost.

Clock accuracy

Source	Accuracy	Use when
Crystal (8/16 MHz)	$\pm 20\text{--}50\text{ ppm}$	Always: UART, USB, SPI, accurate timing
Internal RC (8 MHz)	$\pm 10\%$	Quick prototype only — no communications
32.768 kHz crystal	$\pm 5\text{--}20\text{ ppm}$	RTC wakeup timer, ultra-low-power async

2-stage pipeline and flush

The AVR fetches the next instruction while the current one executes. This gives near 1-instruction-per-cycle throughput.

Pitfall

When a branch is *taken* (if/else, function call) or an interrupt fires, the pre-fetched instruction is wrong and must be discarded — **pipeline flush**: costs 1–4 clock cycles. Branches not taken cost 1 cycle; taken branches cost 2. Minimize branching inside time-critical ISRs. Every ISR entry also flushes the pipeline (then saves PC + SREG).

IoT Hierarchy & Hardware Design

IoT architecture layers

Layer	Term	Hardware example	Role
Thing (sensor)	SS	AT90USB + tachogenerator	Measure, local processing
Thing (actuator)	SA	AT90USB + motor driver	Control the physical world
Edge computing	LCR	Raspberry Pi, gateway	Aggregate, local analytics
Internet gateway	GC	4G router, WiFi AP	Bridge local $\leftrightarrow$ cloud
Cloud	RCR	AWS IoT, Azure	Long-term storage, ML, alerts

What lives where

MCU handles: real-time sensing (10–200 Hz), local control loop, short-term history (seconds). Edge handles: aggregation, medium-term trends. Cloud handles: long-term storage, ML, global analytics — computationally too expensive for the MCU.

Hardware design process — always I/O first

1. Draw a block diagram: MCU in centre; all I/O devices with interface labels; power supply; JTAG/ISP.
2. Select I/O devices: output type, package, voltage, accuracy, current, cost.
3. Tally required MCU I/O (# digital I/O, ADC channels, UARTs, SPIs, I²C, PWM, ISP pins).

4. Select MCU: filter by peripherals first, then memory, clock, voltage, pin count, sleep modes, cost.
5. Design power supply: sum current per rail, add 20 % margin, choose regulator type.

Why I/O first

You cannot choose an MCU until you know what peripherals you need. Choosing the MCU first risks selecting one without enough ADC channels or I²C interfaces. Count the I/O first; the MCU choice follows naturally.

I/O device categories

Category	Examples	Typical interface
Sensors	Temperature, humidity, RPM, pressure	ADC (analog), I ² C, SPI
Actuators	DC motor, servo, solenoid	PWM, GPIO (via driver)
User input	Button, keypad, rotary encoder	GPIO (debounce), interrupt
Display	LCD, OLED, 7-segment	SPI, I ² C, GPIO parallel
Communications	ZigBee, GPS, Bluetooth module	UART (most common)
Storage	SD card, SPI flash, EEPROM	SPI, I ² C

Power supply: LDO vs buck

Type	Pros	Cons	Use when
Linear (LDO)	Low noise, simple, cheap	Inefficient ($P_{loss} = \Delta V \times I$)	Small ΔV , noise-sensitive ADC
Switching (buck)	>80 % efficient	EMI noise, complex layout	Large ΔV , efficiency critical
Buck → LDO	Efficient + clean output	Two stages, more parts	Battery: buck down, LDO filters noise

Software Engineering

4 process activities

Activity	What it means
Specification	Customers + engineers define what the software must do and its constraints. Requirements written <i>before</i> coding.
Development	Software designed and programmed. Architectural design → modularisation → coding.
Validation	Software tested to verify it meets requirements (unit, integration, system testing).
Evolution	Software modified as requirements change. Maintenance often exceeds development cost.

Bug cost pyramid

Requirements bugs: 56 %. Design: 27 %. Code: 7 %. Fix cost multiplier: Requirements (1×) → Design (5×) → Development (10×) → Testing (20×) → Live deployment (200×). **Getting requirements right is the single highest-ROI activity.**

Functional vs non-functional requirements

Functional — WHAT it does	Non-functional — HOW WELL
Measure temperature every 60s	Accuracy: $\pm 0.5^\circ\text{C}$
Transmit wirelessly to farmhouse	Wireless range: >1 km
Store data if communication fails	Operate for 2 years on battery
Detect disk slot crossing event	Slot detection latency: <1 ms
Control motor speed via PWM	Operate -10 to $+50^\circ\text{C}$

Non-functional requirements drive hardware choices

“Battery life 2 years” → average current calculation. “Latency <1 ms” → which timer mode, what clock. “ $\pm 0.5^\circ\text{C}$ accuracy” → which sensor, what ADC resolution and reference.

Hardware vs software partitioning

Function	Hardware	Software	Decision
UART / SPI / I ² C	Built-in peripheral	Bit-bang GPIO	Always hardware: free, deterministic
Switch debouncing	RC + Schmitt trigger	Software timer	Software: cheaper PCB
PWM generation	Timer peripheral	Toggle loop	Always hardware: jitter-free
Precise event timing	Input Capture	Read timer in ISR	Input Capture: hardware-latched, no jitter
CRC / checksum	Hardware CRC (XMEGA)	Software loop	Hardware if available; else software

FSM code skeleton

```

1 typedef enum { IDLE, RAMP_UP, STEADY, STEP_TEST, STOP } state_t;
2 volatile state_t state = IDLE;
3
4 // In main loop - driven by ISR flags
5 void fsm_update(void) {
6     switch (state) {
7         case IDLE:     if (start_cmd) state = RAMP_UP; break;
8         case RAMP_UP:  do_ramp(); if (rpm_ok) state = STEADY; break;

```

```

9      case STEADY:   OCR1A = pid_update(rpm);
10                    if (step_cmd) state = STEP_TEST;
11                    if (fault)   state = STOP; break;
12      case STEP_TEST: log_step(); if (done) state = STEADY; break;
13      case STOP:     OCR1A = 0; if (reset_cmd) state = IDLE; break;
14  }
15 }

```

All Key Formulas — Reference Table

Name	Equation	Description	When to use
Timing			
Clock period	$T_{clk} = 1/f_{cpu}$ $16\text{ MHz} \rightarrow 62.5\text{ ns/cycle}$	Time per CPU clock cycle	Convert clock frequency to real time; used before computing timer ticks
Prescaler tick	$T_{tick} = \text{prescaler}/f_{cpu}$ $/8 \rightarrow 0.5\mu\text{s/count}; \quad /64 \rightarrow 4\mu\text{s/count}$	Real time per timer count	Find how long one timer increment takes; multiply by count for elapsed time
Timers			
CTC OCR	$\text{OCR} = \frac{f_{cpu}}{\text{pre} \times f} - 1$ $200\text{ Hz}, /64 \rightarrow \mathbf{1249}; \quad 1\text{ kHz}, /8 \rightarrow \mathbf{1999}; \quad 8\text{ kHz}, /8 \rightarrow \mathbf{249}$	Compare register value	Set CTC mode interrupt to fire at a target frequency
Verify CTC	$f = \frac{f_{cpu}}{\text{pre} \times (\text{OCR} + 1)}$ $16\text{M}/(64 \times 1250) = 200\text{ Hz} \checkmark$	Actual interrupt frequency	Sanity-check: always verify the OCR gives the intended frequency
ICR time	$\Delta t = \frac{\text{ICR1}_{new} - \text{ICR1}_{old}}{f_{timer}}$ $f_{timer} = 2\text{ MHz } (/8); 400\text{ counts} \rightarrow 200\mu\text{s}$	Real time between events	Convert captured timer counts to real elapsed time
Rev. period	$T_{rev} = 60/\text{RPM}$ $1000\text{ RPM} \rightarrow 60\text{ ms/rev}$	Time per revolution (s)	Basis for slot timing; find expected slot interval at a given speed
Slot time	$T_{slot} = T_{rev} \times f_{slot}$ $8\text{ slots}, 1000\text{ RPM} \rightarrow 7.5\text{ ms/slot}$	Time between slot events	Expected capture interval; compare against ICR delta to verify sync
Speed (ICR)	$v = d/\Delta t$ $\text{Pitch } d = 0.628\text{ mm}, \Delta t = 197\mu\text{s} \rightarrow 3.18\text{ m/s}$	Belt/disk surface speed	Calculate linear speed from encoder pulse pitch and measured period
PWM			
8-bit PWM	$f_{PWM} = \frac{f_{cpu}}{\text{pre} \times 256}$ $/8 \rightarrow 7812\text{ Hz}; \quad /64 \rightarrow 977\text{ Hz}$	PWM output frequency	Find the output frequency when using 8-bit Fast PWM (TOP = 255 fixed)
Var-TOP PWM	$f_{PWM} = \frac{f_{cpu}}{\text{pre} \times (\text{TOP} + 1)}$ $/8, \text{TOP}=\text{ICR1}=399 \rightarrow 5\text{ kHz}$	PWM frequency (custom)	Set a non-standard frequency by loading ICR1 or OCR1A as TOP
Duty cycle	$DC = \text{OCR}/\text{TOP}$ $\text{OCR}=127, \text{TOP}=255 \rightarrow 49.8\%$	Fraction high (0–1)	Controls average power to load; motor speed, LED brightness
Avg voltage	$V_{avg} = V_s \times DC$ $5\text{ V}, DC=50\% \rightarrow 2.5\text{ V}$	Mean output voltage	Average voltage across a filtered load; not valid for reactive loads
ADC			
Step size	$\text{Res} = V_{ref}/1023$ $5\text{ V} \rightarrow 4.88\text{ mV/step}; \quad 2.56\text{ V} \rightarrow 2.50\text{ mV/step}$	Voltage per count (mV)	Minimum measurable voltage change; determines if sensor accuracy is sufficient

Name	Equation	Description	When to use
Voltage	$V = ADC \times V_{ref}/1023$ <i>Count=511, 5 V ref $\rightarrow$ 2.496 V</i>	Measured voltage (V)	Convert raw 10-bit ADC result to physical voltage
Min rate	$f_{s,min} = \frac{RPM_{max}}{60} \times N_{rev}$ <i>1000 RPM, 10/rev $\rightarrow$ 167 Hz; use 200 Hz</i>	Minimum sample rate	ADC must be fast enough to capture every slot event at max speed
ADC clock	$f_{ADC} = f_{cpu}/\text{prescaler}$ (50–200 kHz target) <i>/128 $\rightarrow$ 125 kHz $\checkmark$; /64 $\rightarrow$ 250 kHz (too fast)</i>	ADC clock frequency	Choose prescaler keeping f_{ADC} in valid range; too high degrades accuracy
Conv. time	$t_{conv} = 13/f_{ADC}$ <i>125 kHz $\rightarrow$ 104 μs; period at 200 Hz = 5 ms $\gg$ 104 μs $\checkmark$</i>	Duration of one sample	Must be shorter than the ADC trigger period (timer compare interval)
Max rate	$f_{s,max} = f_{ADC}/13$ <i>125 kHz $\rightarrow$ 9615 Hz max</i>	Upper sample rate limit	Theoretical ceiling; confirm required sample rate is achievable
Nyquist	$f_s \geq 2 \times f_{signal,max}$ <i>100 Hz signal $\rightarrow$ sample at $\geq$ 200 Hz</i>	Min safe sample rate	Avoid aliasing; signal content above $f_s/2$ folds back into the data
UART			
UBRR normal	$UBRR = \frac{f_{cpu}}{16 \times \text{baud}} - 1$ <i>115 200, 16 MHz $\rightarrow$ 8 (error -3.5%: use U2X)</i>	Baud rate register value	Configure UART speed; check error — >2% causes framing errors
UBRR U2X	$UBRR = \frac{f_{cpu}}{8 \times \text{baud}} - 1$ <i>115 200, 16 MHz $\rightarrow$ 16 (error 0.16% $\checkmark$)</i>	UBRR with U2X=1	Set U2X bit in UCSRA to halve the baud divisor; use at 115 200 baud
Throughput	$T_{put} = \text{baud}/10 \text{ B/s}$ <i>115 200 $\rightarrow$ 11 520 B/s; need 400 B/s $\ll$ 11 520 $\checkmark$</i>	Max byte rate	10 bits/frame (start + 8 data + stop); check data rate fits within budget
I²C / TWI			
TWI bit rate	$TWBR = \frac{f_{cpu}/f_{scl}-16}{2}$ <i>100 kHz $\rightarrow$ TWBR=72; 400 kHz $\rightarrow$ TWBR=12</i>	TWBR register value	Set SCL frequency at init; assume prescaler = 1 (TWSR = 0x00)
Memory			
Buffer size	$\text{Buf} = N_{samples} \times \text{bytes/sample}$ <i>200 Hz $\times$ 30 s $\times$ 2 B = 12,000 > 8192: stream via UART</i>	Bytes required	Check if a capture window fits in SRAM; if >8192 stream via UART
EEPROM life	$\text{Life(yr)} = \frac{100,000}{\text{wr/day} \times 365}$ <i>1440/day (every 60 s) $\rightarrow$ 0.19 yr; daily $\rightarrow$ 273 yr</i>	Write endurance (years)	Verify EEPROM survives the write rate over product lifetime
Power & Battery			
Avg current	$I_{avg} = \frac{I_a t_a + I_s t_s}{T_{cycle}}$ <i>Vineyard: (215.4 $\times$ 2 + 0.002 $\times$ 58)/60 = 7.18 mA</i>	Mean current draw (mA)	Weighted average over one active/sleep cycle; input to battery life
Battery life	$\text{Life(h)} = C_{mAh}/I_{avg}$ <i>500 mAh / 7.18 mA = 69.6 h = 2.9 days</i>	Run time (hours)	Estimate operation time; compare against target (e.g. 2 years)
LDO loss	$P_{loss} = (V_{in} - V_{out}) \times I_{load}$	Heat dissipated (W)	Thermal sizing of regulator; justifies switching to a buck if $\Delta V > 2 \text{ V}$

Name	Equation	Description	When to use
$12\text{ V} \rightarrow 5\text{ V}$ at 100 mA : 700 mW wasted as heat			
Hardware Design			
Volt. divider	$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$ $V_{in} = 5\text{ V}, R_1 = R_2 = 10\text{ k}\Omega \rightarrow V_{out} = 2.5\text{ V}$	Scaled output voltage	Scale sensor signal down to ADC input range; biasing; reference voltage
PID (discrete)			
PID output	$u[k] = K_p e + K_i \sum e + K_d (e[k] - e[k-1])$ <i>Clamp $u \in [0, 255]$ for 8-bit OCR; store gains in EEPROM</i>	Control signal $u[k]$	Closed-loop motor/position control; tune K_p, K_i, K_d ; clamp to valid range
Anti-windup	$ \text{integral} \leq I_{max}$ <i>Without clamp: integrator winds up — slow to return to setpoint</i>	Clamp on integrator	Prevent integrator saturation; without this, system takes long to recover

Pitfall

ADC reference: Selecting internal 2.56 V ref in software while 5 V is connected to the AREF pin $\Rightarrow$ internal short circuit, permanent ADC damage.

Pitfall

UART clock: Requires $<2\%$ accuracy. Internal RC oscillator is $\pm 10\%$ — **never** use for UART, USB, or SPI. Always use a crystal. Use U2X mode at 115 200 baud to reduce error from -3.5% to 0.16% .

V_{ref}	Value	Use when
AVCC	5 V	Signal spans full supply range
Int 2.56 V	2.56 V	Low-voltage signal; better resolution for small signals
Ext AREF	User-supplied	Precision reference needed; NEVER mix with internal selection

Data type	Memory
Program code / algorithm	Flash
Const lookup table (sine, calibration curve)	Flash (PROGMEM)
Runtime sensor readings, buffers, stack	SRAM
PID gains / settings — survive power-off	EEPROM
$>8\text{ KB}$ data log	Stream UART or ext SPI flash / SD card

Register Configuration & Code Templates

Timer1 CTC at 200 Hz — ADC trigger

```

1 // CTC mode, prescaler /64, OCR1A=1249 -> 200 Hz
2 TCCR1B = (1<<WGM12)|(1<<CS11)|(1<<CS10);
3 OCR1A = 1249;
4 TIMSK1 = (1<<OCIE1A); // compare A interrupt
5 sei();
6 ISR(TIMER1_COMPA_vect) {
7     ADCSRA |= (1<<ADSC); // trigger conversion
8 }
```

ADC init + result ISR

```

1 void adc_init(void) {
2     ADMUX = (1<<REFS0); // AVCC ref, ch0
3     ADCSRA = (1<<ADEN)|(1<<ADIE) // enable + IRQ
4             |(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0); // /128
5 }
6 ISR(ADC_vect) {
7     buffer[idx++] = ADC; // 10-bit result
8     data_ready = 1;
9 }
```

Timer1 Fast PWM (OC1A)

```

1 // 10-bit Fast PWM, non-inverting, prescaler /8
2 TCCR1A = (1<<COM1A1)|(1<<WGM11)|(1<<WGM10);
3 TCCR1B = (1<<WGM13)|(1<<WGM12)|(1<<CS11);
4 ICR1 = 399; // TOP -> f = 16M/(8*400) = 5 kHz
5 OCR1A = 200; // 50% duty
```

```
6 DDRB |= (1<<PB5); // OC1A pin as output
```

Input Capture (slot timestamps)

```
1 // Timer1 already running. ICP1 pin = input.
2 TCCR1B |= (1<<ICES1); // rising edge trigger
3 TIMSK1 |= (1<<ICIE1); // enable capture interrupt
4 volatile uint16_t ts_prev, ts_new;
5 ISR(TIMER1_CAPT_vect) {
6     ts_new = ICR1; // hardware-latched, no jitter
7     slot_event = 1;
8 }
9 // delta_t = (ts_new - ts_prev) / f_timer_clock
```

PID controller (fixed-point integer)

```
1 int16_t error, integral = 0;
2 volatile int16_t setpoint = 500; // RPM
3
4 uint8_t pid_update(int16_t measured) {
5     error = setpoint - measured;
6     integral += error;
7     // anti-windup clamp
8     if (integral > MAX_I) integral = MAX_I;
9     if (integral < -MAX_I) integral = -MAX_I;
10    int16_t u = (KP * error + KI * integral) >> 8;
11    if (u > 255) u = 255;
12    if (u < 0) u = 0;
13    return (uint8_t)u;
14 }
15 // In 200 Hz flag handler: OCR1A = pid_update(rpm);
```

PROGMEM + EEPROM access

```
1 // PROGMEM: keeps const tables in Flash (saves SRAM)
2 #include <avr/pgmspace.h>
3 const uint8_t sin_tbl[256] PROGMEM = {0, 3, 6, ...};
4 uint8_t v = pgm_read_byte(&sin_tbl[45]);
5
6 // EEPROM: non-volatile settings (read at startup)
7 #include <avr/eeprom.h>
8 uint16_t kp = eeprom_read_word(&kp_addr);
9 // update_word only writes if value actually changed:
10 eeprom_update_word(&kp_addr, new_kp);
11 // Never write EEPROM inside an ISR or fast loop!
```

Watchdog Timer — enable and reset

```
1 #include <avr/wdt.h>
2 int main(void) {
3     wdt_enable(WDTO_250MS); // 250 ms timeout; MCU resets if not kicked
4     // ... other init ...
5     sei();
6     while (1) {
7         wdt_reset(); // kick before timeout; do this every loop
8         if (data_ready) { data_ready = 0; rpm = adc_to_rpm(ADC); }
9         // ...
10    }
11 }
12 // On reset: OCR1A initialises to 0 -> motor stays off. Safe default.
```

SREG save/restore inside an ISR

```
1 // Compiler handles this automatically in avr-gcc,
2 // but understanding it is exam-required.
3 // What hardware + ISR does on every interrupt:
4 // 1. Hardware pushes PC (2 bytes), SP decrements 2.
5 // 2. Hardware clears I-bit (no nested interrupts by default).
6 // 3. CPU jumps to vector address.
7 // 4. ISR prologue: push SREG + all modified registers.
8 // 5. ISR body runs.
9 // 6. ISR epilogue: pop registers + SREG.
10 // 7. RETI: pops PC, restores I-bit. Main() resumes.
11
12 // Manual pattern (when writing ISR in hand-crafted asm or bare C):
13 ISR(TIMER1_COMPA_vect) {
14     uint8_t sreg_save = SREG; // save status flags
15     cli(); // ensure atomicity if needed
16     // ... ISR body ...
17     SREG = sreg_save; // restore flags (including I-bit)
18 }
```

I²C read transaction skeleton

```
1 // I2C sequence: START -> addr+W -> reg -> RESTART -> addr+R -> data -> STOP
2 // TWI registers: TWCR (control), TWDR (data), TWSR (status)
3 #define TWI_START() (TWCR = (1<<TWINT)|(1<<TWSTA)|(1<<TWEN))
4 #define TWI_STOP() (TWCR = (1<<TWINT)|(1<<TWSTO)|(1<<TWEN))
5 #define TWI_SEND(d) do { TWDR=(d); TWCR=(1<<TWINT)|(1<<TWEN); \
```



```

6         while(!(TWCR&(1<<TWINT))); } while(0)
7 #define TWI_READ(ack) (TWCR=(1<<TWINT)|(1<<TWEN)|((ack)?(1<<TWEA):0), \
8         while(!(TWCR&(1<<TWINT))), TWDR)
9
10 uint8_t i2c_read_reg(uint8_t addr, uint8_t reg) {
11     TWI_START();
12     TWI_SEND((addr << 1) | 0); // address + write
13     TWI_SEND(reg); // register pointer
14     TWI_START(); // repeated start
15     TWI_SEND((addr << 1) | 1); // address + read
16     uint8_t data = TWI_READ(0); // read, send NACK (last byte)
17     TWI_STOP();
18     return data;
19 }
20 // Vineyard: i2c_read_reg(0x5F, 0x28) -> HTS221 temp LSB

```

I²C / TWI initialisation

```

1 // f_scl = f_cpu / (16 + 2*TWBR*prescaler)
2 // TWBR = (f_cpu/f_scl - 16) / 2 (prescaler = 1, TWSR = 0x00)
3 // 100 kHz: TWBR = (16 000 000 / 100 000 - 16) / 2 = 72
4 // 400 kHz: TWBR = (16 000 000 / 400 000 - 16) / 2 = 12
5 void twi_init(void) {
6     TWSR = 0x00; // prescaler = 1
7     TWBR = 72; // 100 kHz SCL at 16 MHz
8     TWCR = (1<<TWEN); // enable TWI module
9 }
10 // Requires 4.7 kOhm pull-ups on SDA and SCL to VCC.
11 // Vineyard: three devices on same 2 wires (0x48, 0x5F, 0x68).

```

SPI initialisation (master)

```

1 // Master, CPOL=0 CPHA=0 (Mode 0), f_SCK = f_cpu/16 = 1 MHz
2 // Match CPOL/CPHA to device datasheet - check before using!
3 void spi_init(void) {
4     DDRB |= (1<<PB1)|(1<<PB2)|(1<<PB0); // SCK, MOSI, SS -> output
5     DDRB &= ~(1<<PB3); // MISO -> input
6     SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR0); // enable, master, /16
7 }
8 uint8_t spi_transfer(uint8_t data) {
9     SPDR = data;
10    while (!(SPSR & (1<<SPIF))); // wait for transfer complete
11    return SPDR; // received byte (full-duplex)
12 }
13 // CS pin: assert low BEFORE transfer, deassert high AFTER.
14 // LoRa SX1276: 8 MHz, SD card: start at 400 kHz then switch to 8 MHz.

```

Sleep mode entry (Vineyard power-down)

```

1 #include <avr/sleep.h>
2 // Vineyard: enable WDT or external INT before sleeping, or MCU
3 // sleeps forever. Call this at the end of the main loop.
4 void enter_sleep(void) {
5     set_sleep_mode(SLEEP_MODE_PWR_DOWN); // <1 uA; wake: WDT, INT, TWI
6     sleep_enable();
7     sei(); // I-bit must be set or no interrupt wakes MCU
8     sleep_cpu(); // MCU halts here; ISR runs then resumes below
9     sleep_disable(); // disable sleep after wakeup to prevent accidental re-entry
10 }
11 // Other modes:
12 // SLEEP_MODE_IDLE ~5 mA - timers+UART still run
13 // SLEEP_MODE_ADC_NOISE_REDUCTION ~1 mA - ADC completes, then wakes
14 // SLEEP_MODE_PWR_SAVE <1 uA - Timer2 async (32 kHz RTC) still runs

```

Event-driven main loop template (disk project)

```

volatile uint8_t data_ready = 0, slot_event = 0;
int main(void) {
    timer1_ctc_init(); adc_init(); uart_init(); sei();
    state = IDLE;
    while (1) {
        if (data_ready) { data_ready = 0; rpm = adc_to_rpm(ADC); OCR1A = pid_update(rpm); }
        if (slot_event) { slot_event = 0; process_slot(ts_new); }
        if (tx_pending) { tx_pending = 0; uart_send_frame(); }
        sleep_cpu(); // CPU sleeps; any ISR wakes it
    }
}

```

ISR rules — short version

- Set a flag or copy a byte; do heavy work in `main()`.
- Never call `_delay_ms()`, `printf()`, or EEPROM write.
- Shared variables **must** be **volatile**.
- Guard multi-byte reads in main with `cli()/sei()`.
- Enable in three steps: (1) peripheral IRQ bit, (2) `sei()`, (3) event must occur.
- No priorities in AVR — lower vector address wins.
- Hardware auto-pushes PC; ISR must save/restore SREG.

- After RETI: I-bit automatically restored.

Communication Protocol Selection

Protocol	Wires	Speed	Best for
UART	TX + RX	1 Mbps	PC debug, ZigBee, GPS
SPI	3 + CS	10s MHz	SD card, fast ADC, display
I ² C	SDA+SCL	100k–1 MHz	Multiple slow sensors
USB CDC	2 (diff)	12 Mbps	Fast PC data logging
RS-485	2 (diff)	10 Mbps	Industrial multi-drop
CAN	2 (diff)	1 Mbps	Automotive, safety

Pitfall

Differential (USB, RS-485, CAN) cancels common-mode noise on long cables. Use for cables >1 m or noisy environments. UART, SPI, I²C are single-ended — noise couples directly.

I²C in practice

Pull-ups 4.7 kΩ to VCC (open-drain bus). Vineyard: temp (0x48), humidity (0x5F), RTC (0x68) all share 2 wires. GPIO-controlled pull-ups can be disabled during sleep to save power.

SPI in practice

Master drives SCK + CS (active-low); full-duplex (MOSI and MISO simultaneously). Must clock out a dummy byte to receive data. Match CPOL/CPHA to device datasheet. Typically 4–8 MHz on AT90USB.

USB CDC advantage

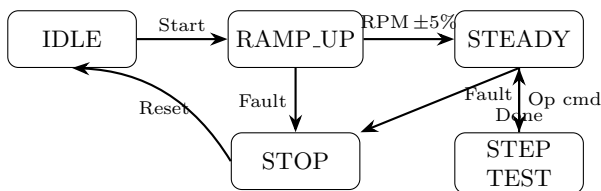
AT90USB1287 has native USB hardware. CDC library makes MCU appear as a COM port. 30 KB data arrives in ≈22 ms over USB vs ≈2.6 s over UART.

Software Architecture & FSM Diagrams

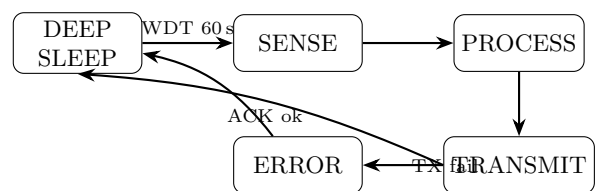
Pattern	Pro	Con
Polling loop	Simple code	Blocks CPU; misses fast events
Interrupt-driven	Responsive; sleep between	Race conditions on shared data
FSM	Clear operational states	More structural discipline
RTOS	Multiple independent tasks	Overhead; not needed here

Best choice: Interrupt-driven + FSM combined. ISRs set flags; main loop reads flags and drives FSM state transitions; CPU sleeps with `sleep_cpu()`.

Defensive technique	Implementation
Watchdog Timer	<code>wdt_reset()</code> every loop; auto-resets MCU on hang
POST	Check SRAM, EEPROM checksum, sensor comms at startup
Checksums	Verify EEPROM on read; load safe defaults on mismatch
Boundary checks	Validate all values before use (<code>if(rpm>1100) rpm=1000</code>)
Re-write registers	Rewrite critical registers periodically in main loop
<code>cli()/sei()</code>	Guard multi-byte shared variable accesses
Unused vectors	Map to safe handler (never random Flash)



Disk Sync FSM



Vineyard IoT FSM

Modular design (3-level hierarchy)

Modules (3–5 per project): Disk: Sensor, Control, Actuator, Capture, Comms. **Source files** (1–2/module): `.c/.h` pairs; anything not in the header is private. **Functions** (3–6/file): each does one small, testable task.

Driver level	Example
Low	<code>SPDR = data;</code>
Mid	<code>spi_transfer(uint8_t data)</code>
High	<code>readSPIReg(device, addr)</code>

High-level hides chip-select timing and register formats. Caller just says “read register X of device Y”.

Worked Examples — Q1 to Q5

Worked: Q1 — Disk speed sensing: full hardware + software specification

Requirement: 10 samples/revolution, max 1000 RPM.

1. **Sample rate:** $f = (1000/60) \times 10 = 166.7 \text{ Hz} \rightarrow$ use **200 Hz**.

2. **Timer CTC:** prescaler 64, OCR1A = $16\text{M}/(64 \times 200) - 1 = \mathbf{1249}$. Period = 5 ms. Timer1 Compare A ISR fires at 200 Hz and triggers ADC.

3. **ADC:** AVCC = 5 V, 10-bit. Res = $4.88 \text{ mV/step} = 0.98 \text{ RPM/step} \checkmark$. Prescaler 128: $f_{ADC} = 125 \text{ kHz}$, $t_{conv} = 104 \mu\text{s} < 5 \text{ ms.} \checkmark$

4. **Buffer check:** $200 \times 30 \times 2 = 12,000 \text{ bytes} > 8192 \Rightarrow$ **stream via UART**.

5. **UART check:** 11 520 bytes/s available. Required: $200 \times 2 = 400 \text{ bytes/s.} \checkmark$

Worked: Q2 — Vineyard: I²C vs SPI vs UART justification

Temp + Humidity + RTC $\rightarrow$ I²C. All three are I²C-compatible (HTS221 at 0x5F, DS3231 at 0x68). I²C uses only 2 wires regardless of device count; SPI would require 3 separate CS lines — GPIO penalty grows with each sensor. 100 kHz is ample (sensors update at $< 1 \text{ Hz}$). Pull-ups 4.7 k Ω ; GPIO-controlled to save sleep-mode power.

ZigBee module $\rightarrow$ UART. XBee presents a UART interface. UART is point-to-point matching MCU-to-module. MCU sends ASCII/binary frames at 9600–115 200 baud; the module handles the full 802.15.4 MAC layer internally. ZigBee TX current = 205 mA — sleep between transmissions is essential.

Worked: Q3 — Disk sync: functional and non-functional requirements

Functional (WHAT the system does):

1. Measure disk speed (0–1000 RPM) continuously.
2. Control motor speed via PWM to achieve user-set target.
3. Detect disk slot crossing events.
4. Log speed, duty cycle, and timestamp during step-response tests.
5. Transmit logged data to a PC.
6. Apply step change in target speed on operator command.

Non-functional (HOW WELL):

1. Sampling rate $\geq 167 \text{ Hz}$ at 1000 RPM.
2. Slot timing accuracy $< 10 \mu\text{s}$ (requires hardware Input Capture, not software polling).
3. Control loop update rate $\geq 200 \text{ Hz}$ for stable PID.
4. Real-time data streaming (12 KB log $> 8 \text{ KB}$ SRAM, must stream).
5. PID gains survive power-off (stored in EEPROM).
6. Motor stops safely if watchdog expires.

Worked: Q4 — Vineyard battery life

Given: 60 s cycle, 2 s active, 58 s sleep, 500 mAh battery.

Active: MCU 10 + ZigBee 205 + sensors 0.4 = 215.4 mA. Sleep: MCU + ZigBee + sensors off = 0.002 mA.

$$I_{avg} = \frac{215.4 \times 2 + 0.002 \times 58}{60} = \frac{430.9}{60} = \mathbf{7.18 \text{ mA}} \quad \text{Life} = \frac{500}{7.18} = \mathbf{2.9 \text{ days}}$$

To extend to 30 days (10-min cycle): $I_{avg} = (215.4 \times 2 + 0.002 \times 598)/600 = \mathbf{0.72 \text{ mA}} \Rightarrow$ Life = 694 h = **29 days**.

For 2-year target: Required $I_{avg} = 500/17520 = 0.0285 \text{ mA} \Rightarrow$ only $\approx 8 \text{ ms}$ active/minute. Needs solar charging or larger battery.

Worked: Q5 — Disk project: software architecture and modules

Architecture: Event-driven + FSM. ISRs set flags; main loop processes them and drives FSM.

Modules: `adc.c` (Sensor: read voltage $\rightarrow$ RPM), `pid.c` (Control: PID $\rightarrow$ OCR), `motor.c` (Actuator: Timer1 Fast PWM), `capture.c` (Events: Input Capture $\rightarrow$ timestamps), `uart.c` (Comms: streaming).

Interrupts: Timer1 CTC (200 Hz) $\rightarrow$ ADC trigger; ADC complete $\rightarrow$ store result, set `data_ready`; Input Capture $\rightarrow$ copy ICR1, set `slot_event`.

FSM: IDLE $\rightarrow$ RAMP.UP $\rightarrow$ STEADY $\rightarrow$ STEP.TEST $\rightarrow$ STOP.

EEPROM: K_p , K_i , K_d stored; reloaded on every reset — tuning survives power cycles.

Extended Scenarios — Full System Design

Worked: Industrial Conveyor Belt Speed Control

System overview. A manufacturing conveyor belt must maintain constant speed (0–2 m/s) under varying load. An optical encoder (500 pulses/rev, 0.1 m diameter drum) measures belt speed. A DC motor is driven via an H-bridge using PWM. RS-485 connects to a factory PLC for supervisory control; a local UART panel serves the operator. An ACS712 current sensor on the ADC detects overcurrent and belt jams.

1. Physical quantities and data flow

- Belt speed $\rightarrow$ encoder pulses $\rightarrow$ Timer1 Input Capture (ICR1) $\rightarrow$ speed calculation $\rightarrow$ PID $\rightarrow$ PWM duty $\rightarrow$ H-bridge $\rightarrow$ motor.
- Motor current $\rightarrow$ ACS712 (100 mV/A, centred at 2.5 V) $\rightarrow$ ADC channel 1 $\rightarrow$ overcurrent check.
- PLC commands (setpoint, start/stop) $\rightarrow$ RS-485 frame $\rightarrow$ UART peripheral $\rightarrow$ FSM.

2. Speed sensing calculation

At 2 m/s: drum circumference = $\pi \times 0.1 = 0.314$ m, so rev/s = $2/0.314 = 6.37$ and pulses/s = $6.37 \times 500 = 3185$ Hz.

Timer1 Input Capture, prescaler 8 ($t_{tick} = 0.5 \mu\text{s}$). Counts per pulse at full speed:

$$\frac{16\,000\,000}{8 \times 3185} = 628 \text{ counts}$$

Speed from ICR1 (pulse pitch $d = \pi \times 0.1/500 = 0.628$ mm):

$$v = \frac{d}{(ICR1_{new} - ICR1_{prev}) \times 0.5 \mu\text{s}}$$

At minimum speed (0.1 m/s): 318.5 pulses/s, period = $3140 \mu\text{s} = 6280$ timer counts — well within 16-bit timer range (65535 max). ✓

3. Timer conflict and resolution

Input Capture requires Timer1 in Normal mode. CTC mode for the 100 Hz PID interrupt cannot share Timer1 simultaneously. **Solution:** use Timer3 (also 16-bit) in CTC for the PID loop.

$$OCR3A = \frac{16\,000\,000}{64 \times 100} - 1 = 2499 \quad (100 \text{ Hz PID update})$$

4. Current sensing and fault detection

ACS712 30 A: $V_{out} = 2.5 + 0.1 \times I_{motor}$ (centred at 2.5 V = 0 A).

$$ADC = \left\lfloor \frac{V_{out} \times 1023}{5} \right\rfloor$$

At 5 A: $V = 3.0 \text{ V} \rightarrow ADC = 614$. Overcurrent threshold (10 A): $V = 3.5 \text{ V} \rightarrow ADC = 716$.

On overcurrent trigger: set OCR to 0 (motor off) immediately in the ADC ISR flag handler, then transition FSM to FAULT.

Belt jam detection: encoder pulses stop (capture delta exceeds timeout) while motor current is high $\Rightarrow$ JAM fault.

Encoder failure: pulses stop while current is at no-load level $\Rightarrow$ ENCODER_FAIL fault.

5. Communications: RS-485 vs UART

- **RS-485 to PLC:** Differential signalling, immune to industrial EMI. Supports multi-drop (up to 32 nodes, up to 1200 m cable). Half-duplex — a GPIO pin controls DE/RE on the MAX485 transceiver (high = transmit, low = receive). Frame structure: [STX] [id] [cmd] [data2] [CRC16] [ETX].
- **UART to operator panel:** Point-to-point, short run (<2 m), simpler. 115 200 baud $\rightarrow$ 11 520 bytes/s.
- Data rate check: $100 \text{ Hz} \times 5 \text{ params} \times 2 \text{ bytes} = 1000 \text{ bytes/s} \ll 11\,520$. ✓
- Why not I²C or SPI for the PLC? Neither supports the distances or node counts required in an industrial environment.

6. Memory sizing

SRAM buffer: $100 \text{ Hz} \times 5 \text{ params} \times 2 \text{ bytes} \times 4 \text{ s} = 4000 \text{ bytes}$. Fits with headroom for stack and globals. A 4-second window is sufficient for step-response logging.

EEPROM: speed setpoints, K_p , K_i , K_d , overcurrent threshold = 12 bytes. At 10 operator adjustments/day:

$$\text{EEPROM life} = \frac{100\,000}{10 \times 365} = 27.4 \text{ yr} \quad \checkmark$$

7. State machine

IDLE (motor off, PLC awaited) $\rightarrow$ **RAMP** (gradually increase PWM to setpoint speed) $\rightarrow$ **RUN** (PID active, current + encoder monitored) $\rightarrow$ **FAULT** (overcurrent or jam: motor off, alert PLC) $\rightarrow$ **IDLE** on operator reset.

An **ESTOP** state (hardware E-stop input, GPIO interrupt) forces motor off regardless of current FSM state and latches until a physical key-reset.

8. Software modules and interrupts

Modules: `encoder.c` (Input Capture ISR, speed computation), `pid.c` (PID algorithm, setpoint), `motor.c` (Timer1 Fast PWM, H-bridge direction GPIO), `current.c` (ADC channel 1, fault thresholds), `comms.c` (RS-485 framing + direction GPIO, UART HMI), `fsm.c` (state transitions, fault logging to EEPROM).

Interrupts: Timer3 Compare A (100 Hz PID flag), Timer1 Input Capture (encoder pulse), ADC complete (current sample), UART RX complete (RS-485 frame), INT0 (E-stop button — highest priority by vector address).

9. Defensive design

- **Watchdog (250 ms):** If PID loop hangs, WDT resets MCU. Motor driver re-initialises to OCR=0 on startup — motor cannot run uncontrolled after reset.
- **Register re-write:** TCCR1A, OCR1A, and H-bridge direction GPIO rewritten every 10 PID cycles to recover from EMI corruption (common in industrial environments near variable-frequency drives).
- **EEPROM checksum:** CRC8 of all settings stored alongside them. Mismatch $\Rightarrow$ load safe defaults (lowest speed, conservative current limit) and alert PLC.
- **Boundary checks:** PID output clamped to $[0, 255]$; speed setpoint validated $\in [0, \text{RPM}_{max}]$ before storing.

Worked: Solar-Powered Wildlife GPS Tracker

System overview. A compact device attached to wildlife logs GPS position, temperature, and movement. Powered by a 2000 mAh LiPo with a small solar panel. Position is transmitted via LoRa SX1276 (SPI) to a distant base station and logged to an SD card (SPI). Motion is detected by an MPU-6050 accelerometer (I²C, 0x68). Temperature is read from an HTS221 (I²C, 0x5F). At rest: wake every

5 minutes (WDT). On motion: wake every 1 minute.

1. Physical quantities and data flow

- GPS position (NEO-6M, NMEA UART at 9600 baud) → UART RX ISR → 100-byte parse buffer → lat/lon extraction.
- Motion event (MPU-6050 INT pin, active-low) → INTO ISR → sets short-wake flag (1-minute cycle).
- Temperature (HTS221 via I²C) → 16-bit signed value.
- All data → LoRa 18-byte packet (uplink) + 32-byte SD log entry (local).

2. Power budget and battery life

Active phase components per 5-minute (300 s) cycle (warm GPS fix assumed after deployment):

Phase	Duration	Current	Charge
GPS acquisition	1.0 s	60 mA	60 mAs
Sensor read (I ² C)	0.5 s	12 mA	6 mAs
LoRa TX (SF7/125k)	0.2 s	120 mA	24 mAs
SD card write	0.5 s	18 mA	9 mAs
Deep sleep	297.8 s	1.0 mA	297.8 mAs
Total / cycle	300 s	—	396.8 mAs

$$I_{avg} = \frac{396.8 \text{ mAs}}{300 \text{ s}} = 1.32 \text{ mA} \quad \text{Life} = \frac{2000 \text{ mAh}}{1.32 \text{ mA}} = 1515 \text{ h} = \mathbf{63 \text{ days (no solar)}}$$

With solar (50 mm×30 mm cell, effective average ≈8 mA over 24 h accounting for weather): Net consumption = 1.32 − 8 = −6.68 mA ⇒ net charging in typical conditions. In heavy overcast (effective 1 mA solar): net = 0.32 mA ⇒ life = 2000/0.32 = 260 days.

Dominant power consumer: LoRa TX at 120 mA for 200 ms is 24 mAs/cycle, far exceeding the sleep current (1 mA × 297 s = 297 mAs is large in total, but the *rate* during sleep is trivial). Reducing LoRa TX frequency or using SF7 instead of SF12 gives the biggest power gains.

3. GPS UART configuration

9600 baud: UBRR = 16 000 000/(16 × 9600) − 1 = **103**.

NMEA sentences (GGA + RMC) ≈ 160 bytes/s at 1 Hz ≪ 960 bytes/s available throughput. ✓

UART RX ISR accumulates characters into a 100-byte ring buffer until \$ (start of sentence) or \r\n (end) is detected, then sets a `gps_line_ready` flag for main-loop parsing.

Cold-start GPS fix can take 30 s (no almanac) to 60 s (very cold). Enforce a 60-second timeout: if no valid fix, log a failure entry and return to sleep immediately to conserve battery.

4. LoRa SX1276 SPI and packet design

SX1276 via SPI at 8 MHz. Packet payload:

Field	Bytes
Device ID	2
Timestamp (epoch seconds)	4
Latitude (fixed-point ×10 ⁶)	4
Longitude (fixed-point ×10 ⁶)	4
Temperature (×10, signed)	2
Status flags (GPS valid, motion active)	2
Total	18

TX time at SF7/BW125 kHz: ≈41 ms (range ≈2 km open). At SF12/BW125 kHz: ≈2.8 s (range ≈20 km), but 68× more airtime energy and subject to regional 1% duty-cycle limits.

Trade-off: SF7 for study areas with base station nearby (less power, less interference); SF12 for remote wilderness deployment (longer range, more power).

SX1276 requires >20 register writes at startup. Store the register-address/value pairs as a `const` table in PROGMEM — saves ≈40 bytes of SRAM.

5. Shared SPI bus: LoRa and SD card

Both LoRa (SX1276) and the SD card use SPI. They share MOSI, MISO, and SCK; each has its own active-low CS pin (GPIO).

Rule: assert only one CS at a time. Before any LoRa transaction: ensure SD_CS is high. Before any SD transaction: ensure LORA_CS is high. A wrapper function `spi_select(device)` enforces this.

SD log entry (32 bytes): timestamp + lat + lon + temperature + status. At 12 readings/hour: 9216 bytes/day. A 4 GB SD card stores ≈434 000 days of data — effectively unlimited.

6. Voltage compatibility issue

AT90USB1287 operates at 5 V; GPS (NEO-6M), LoRa (SX1276), MPU-6050, and HTS221 are all 3.3 V logic devices.

Required: 5 V → 3.3 V level shifters on SPI lines (MOSI, SCK, CS) and I²C lines (SDA, SCL). Use 74LVC245 for unidirectional SPI, BSS138 MOSFETs for bidirectional I²C.

Better alternative for a new design: choose a 3.3 V MCU (e.g., ATmega328P at 3.3 V/8 MHz) and eliminate all level shifting. This also reduces active power from ≈15 mA to ≈4 mA.

7. State machine

DEEP_SLEEP (WDT timer: 5 min or 1 min if motion flag set) → **WAKE_CHECK** (read MPU-6050, update wake interval) → **GPS_ACQ** (power GPS module, await NMEA fix, 60 s timeout) → **TRANSMIT** (LoRa TX + 3 retries) → **LOG** (SD write) → **DEEP_SLEEP**.

On GPS timeout: skip TRANSMIT and LOG data normally (mark GPS fields as invalid), log failure flag, return to DEEP_SLEEP.

On LoRa TX failure after 3 retries: log failure to SD, continue to LOG state anyway (data not lost locally).

8. Software modules and interrupts

Modules: `gps.c` (NMEA parser, UART RX buffering), `lora.c` (SX1276 SPI driver, packet builder, TX/retry logic), `sd.c` (SPI FAT driver, CSV writer), `sensors.c` (I²C driver for MPU-6050 + HTS221), `power.c` (WDT config, sleep modes, peripheral power switching), `fs.c` (state machine, timer management).

Interrupts: WDT interrupt (wakeup from DEEP_SLEEP), INT0 (MPU-6050 motion, set short-wake flag), UART RX (GPS NMEA byte accumulation).

9. EEPROM use

Device ID (2), LoRa frequency band (4), spreading factor (1), sleep interval (1): = 8 bytes. Written at deployment only.

$$\text{EEPROM life} = \frac{100\,000}{1 \times 365} = 273 \text{ yr} \quad \checkmark$$

Store CRC16 of all EEPROM settings. On startup: verify CRC. Mismatch $\Rightarrow$ load safe defaults (SF7, 5-min interval, known frequency) and log an alert via LoRa.

10. Defensive design

- **GPS watchdog:** Software counter; if no valid NMEA fix in 60s, power off GPS and abort phase.
- **LoRa retries:** 3 attempts with 500 ms back-off. After 3 failures, log TX error to SD and continue.
- **SD absent:** If SD init fails at startup, set `sd.ok = 0`; continue LoRa-only mode. Flag stored in EEPROM for diagnostics.
- **Brown-out detector:** Enable BOD at 2.7 V. If LiPo drops below this, MCU resets cleanly rather than corrupting an SD write mid-transaction.
- **Main WDT (8 s):** Reset in main loop. If GPS or I²C bus hangs (common failure mode: sensor holds SDA low), WDT resets MCU and restores safe state.

Practice Questions — Q1 to Q15

Exam Q & A Q1 — ADC resolution

Q: LM35 on 10-bit ADC, 5 V ref. Resolution in °C? Sufficient for $\pm 0.5^\circ\text{C}$?

A: LM35: 10 mV/°C. Res = $5/1023 = 4.88 \text{ mV} = 0.49^\circ\text{C/step}$. Borderline. Better: use 1.1 V internal ref after amplification ($\rightarrow 0.1^\circ\text{C/step}$), or digital DS18B20 (12-bit, 0.0625 °C).

Exam Q & A Q2 — volatile

Q: Why must shared ISR/main variables be `volatile`?

A: The compiler may cache the value in a CPU register, never re-reading it from memory. `volatile` forces every access to go to memory, ensuring main sees every update made by the ISR.

Exam Q & A Q3 — OCR calculation

Q: OCR1A for 1 kHz CTC, 16 MHz, prescaler 8.

A: OCR1A = $16\text{M}/(8 \times 1000) - 1 = 1999$. Verify: $16\text{M}/(8 \times 2000) = 1000 \text{ Hz}$. $\checkmark$

Exam Q & A Q4 — EEPROM suitability

Q: Vineyard logs 30-day average temp, updating daily. Is EEPROM suitable?

A: Life = $100,000/(1 \times 365) = 273 \text{ yr}$. Yes. Use `eeeprom_update_word()` which only writes if value changed.

Exam Q & A Q5 — PROGMEM

Q: Why does Harvard architecture require PROGMEM for constant tables?

A: Standard C copies constants from Flash to SRAM at startup, consuming precious SRAM. PROGMEM keeps data in Flash; read it with `pgm_read_byte()`. A 256-byte sine table without PROGMEM wastes 3% of SRAM.

Exam Q & A Q6 — Input Capture fault

Q: Input Capture ISR configured but never fires. Two likely causes?

A: (1) `sei()` not called — global I-bit not set. (2) ICIE1 bit in TIMSK1 not set, or ICP1 pin not configured as input.

Exam Q & A Q7 — 8-bit Timer0 PWM at 1 kHz

Q: Timer0, 16 MHz, prescaler for $\approx 1 \text{ kHz}$ PWM? 25% duty OCR0B?

A: Prescaler 64: $16\text{M}/(64 \times 256) = 976.6 \text{ Hz}$. OCR0A = 255 (TOP). 25%: OCR0B = $0.25 \times 255 = 64$.

Exam Q & A Q8 — Memory types

Q: When to store a lookup table in Flash vs SRAM?

A: Flash (PROGMEM): constant at runtime — saves SRAM. SRAM: only if the table must be modified at runtime. EEPROM: if it must survive power-off and be writable (e.g. calibration, <100k writes).

Exam Q & A Q9 — Block diagram: ADC to SPI

Q: LM35 → ADC → MCU → SPI → second MCU. Label all interfaces.

A: LM35 (10 mV/°C, analog) $\xrightarrow{\text{ADC0}}$ AT90USB1287 $\xrightarrow{\text{MOSI/MISO/SCK/CS}}$ Slave MCU. ADC gives 10-bit result; master clocks 2 bytes via SPDR. Match CPOL=0, CPHA=0. Common ground required.

Exam Q & A Q10 — ADC value calculation

Q: 10-bit ADC, $V_{ref} = 3.3\text{ V}$, input 1.65 V. Result? At +100 mV drift?

A: $\lfloor 1.65 \times 1023 / 3.3 \rfloor = 511$. Drift: $\lfloor 1.75 \times 1023 / 3.3 \rfloor = 542$.

Exam Q & A Q11 — SPI vs I²C comparison

Q: One advantage and one disadvantage of each.

A: **SPI** + full-duplex, fast. **SPI** – needs 1 CS per device. **I²C** + only 2 wires for ≤ 127 devices. **I²C** – half-duplex, 100–400 kHz, needs pull-ups.

Exam Q & A Q12 — SRAM buffer check

Q: 20 samples/s, 60 s, 4 bytes each. Fits in SRAM?

A: $20 \times 60 \times 4 = 4800$ bytes < 8192. Fits. At 50 samples/s: $12,000 > 8192$ — stream via UART (200 bytes/s needed $\ll$ 11 520 available).

Exam Q & A Q13 — 3.3 V MCU driving 5 V LED strip

Q: 3.3 V MCU must drive WS2812B data line (5 V V_{IH}).

A: WS2812B $V_{IH} > 3.5\text{ V}$; 3.3 V MCU output unreliable. Use **74AHCT125** level-shifter: accepts 3.3 V in ($V_{IH} \approx 2\text{ V}$), drives 5 V out. Power 74AHCT125 from 5 V.

Exam Q & A Q14 — Interrupts vs polling

Q: When is interrupt better than polling? Give one example each.

A: **Interrupt:** event is infrequent/unpredictable; missing it loses data — UART byte arriving at 115 200 baud. **Polling:** event is continuous and predictable; CPU has nothing else to do — free-running ADC in a tight loop.

Exam Q & A Q15 — USB data logger: FSM + timer + ADC config

Q: 8 kHz analog signal, streamed via USB CDC. Design FSM + config.

A: **FSM:** IDLE → SAMPLING (Timer0 CTC 8 kHz triggers ADC; ADC ISR fills double buffer) → TRANSMIT (USB CDC sends 512-byte packet when buffer full) → IDLE on stop.

Timer0: prescaler 8, $\text{OCR0A} = 16\text{M} / (8 \times 8000) - 1 = 249$.

ADC: prescaler 128 → 125 kHz; $t_{conv} = 104\text{ }\mu\text{s} < 125\text{ }\mu\text{s}$ period. ✓

SRAM: 2×512 buffers + 2 KB USB stack = 3 KB < 8 KB. ✓ USB 12 Mbps $\gg$ 16 KB/s data.

Exam Q & A Q16 — Watchdog timer

Q: What is a watchdog timer and why is it essential in safety-critical embedded systems?

A: Hardware counter that resets the MCU if software does not periodically clear it (`wdt.reset()` in the main loop). If code hangs due to an infinite loop, EMI corruption, or stack overflow, the WDT counter expires and forces a hardware reset, returning to a safe state. In any system controlling a real-world actuator (motor, valve, heater), a software lockup could leave the actuator in a dangerous state indefinitely. The WDT ensures automatic recovery. Every safety-critical embedded system must have one.

Additional Scenario — USB Audio / Data Logger

USB Data Logger

Analog signal (microphone or sensor) sampled at 8 kHz (10-bit, ADC0), streamed to PC via AT90USB1287 native USB in CDC mode. Button starts/stops logging. No RTOS needed.

Data flow: Mic/preamp → ADC0 → double buffer (2×256 samples = 512 bytes each) → USB CDC endpoint → PC receives continuous 16 KB/s stream.

Constraints:

- ISR has 125 μs max (104 μs conv + 21 μs overhead). Keep minimal.
- 44 kHz audio: $2 \times 1024 + 4\text{ KB}$ USB stack $\approx 6\text{ KB}$ — approaching limit.
- USB always active: no sleep. Battery use needs USB suspend + GPIO wakeup.

Quick Reference — All Key Formulas

Name	Formula	Key result
Clock period	$T_{clk} = 1/f_{cpu}$	62.5 ns @ 16 MHz
CTC OCR	$OCR = f_{cpu}/(\text{pre} \times f) - 1$	/64, 200 Hz → 1249
Var-TOP PWM	$f = f_{cpu}/(\text{pre} \times (\text{TOP} + 1))$	/8, TOP=399 → 5 kHz
Duty cycle	$DC = OCR/\text{TOP}$	127/255 → 50%
Avg voltage	$V_{avg} = V_s \times DC$	5 V, 50% → 2.5 V
ICR time	$\Delta t = (ICR1_{new} - ICR1_{old})/f_{timer}$	/8 → ÷2 MHz
T_{rev}	$T_{rev} = 60/RPM$	1000 RPM → 60 ms
T_{slot}	$T_{slot} = T_{rev} \times f_{slot}$	8 slots → 7.5 ms
Speed from ICR	$v = d/\Delta t$	0.628 mm, 197 μs → 3.18 m/s
ADC step	$\text{Res} = V_{ref}/1023$	5 V → 4.88 mV
ADC voltage	$V = ADC \times V_{ref}/1023$	511, 5 V → 2.50 V
ADC min rate	$f_s = RPM_{max}/60 \times N$	1000 RPM, 10 → 167 Hz
ADC conv time	$t_{conv} = 13/f_{ADC}$	125 kHz → 104 μs
Nyquist	$f_s \geq 2 \times f_{max}$	100 Hz sig → ≥200 Hz
UBRR normal	$UBRR = f_{cpu}/(16 \times \text{baud}) - 1$	115k, 16M → 8 (−3.5%)
UBRR U2X	$UBRR = f_{cpu}/(8 \times \text{baud}) - 1$	115k, 16M → 16 (0.16%)
Throughput	$T_{put} = \text{baud}/10 \text{ B/s}$	115k → 11 520 B/s
Buffer	$N \times \text{bytes/sample}$	>8192 B: stream
EEPROM life	$100,000/(\text{wr/day} \times 365)$	daily → 273 yr
Avg current	$I_{avg} = (I_{ata} + I_{sts})/T$	Vineyard → 7.18 mA
Battery life	C_{mAh}/I_{avg}	500/7.18 → 2.9 days
LDO loss	$P = (V_{in} - V_{out}) \times I$	12V→5V, 100 mA → 700 mW
Voltage divider	$V_{out} = V_{in} \times R_2/(R_1 + R_2)$	$R_1 = R_2 = 10 \text{ k}\Omega \rightarrow V_{out} = V_{in}/2$
TWI bit rate	$TWBR = (f_{cpu}/f_{scl} - 16)/2$	100 kHz → 72; 400 kHz → 12
PID output	$u = K_p e + K_i \sum e + K_d \Delta e$	clamp [0, 255]

AT90USB1287 specs

Flash	128 KB
SRAM	8 KB
EEPROM	4 KB, 100k cyc
ADC	10-bit, ≈9.6 kHz max
Sleep	< 1 μA (power-down)
Active	10–20 mA @ 16 MHz

Abbreviations & Unit Relationships

Abbreviations

Abbrev.	Meaning	Architecture area
CPU & Architecture		
MCU	Microcontroller Unit — CPU, peripherals, memory all on one chip	CPU / Architecture
MPU	Microprocessor Unit — CPU core only; needs external memory and I/O	CPU / Architecture
RISC	Reduced Instruction Set Computer — mostly single-cycle instructions	CPU / Architecture
Harvard	Separate address spaces for program (Flash) and data (SRAM) memory	CPU / Architecture
SREG	Status Register — CPU flag bits: carry, zero, overflow, I-bit (interrupt enable)	CPU / Architecture
PC	Program Counter — address of next instruction to fetch	CPU / Architecture
SP	Stack Pointer — top of stack; decrements on PUSH, increments on POP	CPU / Architecture
DMA	Direct Memory Access — peripheral-to-memory transfer without CPU involvement	CPU / Architecture
ISP	In-System Programming — program Flash/EEPROM while MCU is in the circuit	CPU / Architecture
JTAG	Joint Test Action Group — debug and boundary-scan hardware interface	CPU / Architecture
Interrupts & Software		
ISR	Interrupt Service Routine — handler executed when an interrupt fires	Interrupts
CLI	Clear global Interrupt enable — disables all maskable interrupts (<code>cli()</code>)	Interrupts
SEI	Set global Interrupt enable — enables all maskable interrupts (<code>sei()</code>)	Interrupts
RETI	Return from Interrupt — pops PC, restores I-bit, resumes <code>main()</code>	Interrupts
FSM	Finite State Machine — software structure for sequential logic	Software
PID	Proportional–Integral–Derivative — closed-loop control algorithm	Software
CRC	Cyclic Redundancy Check — error-detection checksum; used to verify EEPROM contents	Software
Memory		
SRAM	Static RAM — volatile; stores globals, stack, and buffers (8KB on AT90USB)	Memory
EEPROM	Electrically Erasable Programmable ROM — non-volatile; 100 000 write cycles max	Memory
Flash	Non-volatile program memory — read-only at runtime; holds code and PROGMEM data	Memory
PROGMEM	AVR GCC attribute — keeps constants in Flash; read with <code>pgm_read_byte()</code>	Memory
Defensive Design		
WDT	Watchdog Timer — hardware counter that resets MCU if not kicked before timeout	Defensive Design
BOD	Brown-Out Detector — resets MCU if VCC drops below a configured threshold	Defensive Design
POST	Power-On Self Test — startup checks (SRAM, EEPROM CRC, sensor comms)	Defensive Design
Timer Peripherals (AVR)		
CTC	Clear Timer on Compare — timer mode where counter resets at OCR; ISR fires at exact frequency	Timer (AVR)
OCR	Output Compare Register — threshold for CTC match or PWM duty cycle	Timer (AVR)
ICR	Input Capture Register — hardware-latches timer value on ICP pin event; no software jitter	Timer (AVR)
ICP	Input Capture Pin — GPIO pin that triggers a hardware input capture event	Timer (AVR)
TCCR	Timer/Counter Control Register — timer mode, waveform generation, and prescaler	Timer (AVR)
TIMSK	Timer Interrupt Mask Register — enables individual timer interrupt sources	Timer (AVR)
OC1A	Output Compare pin 1A — hardware PWM output driven by Timer1 compare match	Timer (AVR)
ADC (AVR)		
ADC	Analogue-to-Digital Converter — converts input voltage to a 10-bit integer	ADC

Abbrev.	Meaning	Architecture area
DAC	Digital-to-Analogue Converter — not on AT90USB; emulated via PWM + low-pass filter	ADC / Output
ADMUX	ADC Multiplexer Register — selects input channel and voltage reference source	ADC (AVR)
ADCSRA	ADC Control & Status Register A — enable, start, prescaler, interrupt enable/flag	ADC (AVR)
AVCC	ADC supply voltage pin — connect to VCC for 5 V ADC reference	ADC (AVR)
AREF	External ADC reference pin — supply precision voltage; never connect when internal ref active	ADC (AVR)
PWM & Motor		
PWM	Pulse-Width Modulation — output switches at fixed frequency; duty cycle controls power	PWM / Motor
DC	Duty Cycle — fraction of period the output is high (0–100%)	PWM / Motor
VFD	Variable-Frequency Drive — industrial AC motor controller; major source of EMI	PWM / Motor
UART		
UART	Universal Asynchronous Receiver-Transmitter — serial comms with no clock wire	UART
UBRR	UART Baud Rate Register — value that sets the serial bit clock rate	UART (AVR)
UCSRA	UART Control & Status Register A — U2X bit, framing error flag, data register empty	UART (AVR)
U2X	Double-speed UART mode — halves baud divisor; reduces clock error at 115 200 baud	UART (AVR)
SPI		
SPI	Serial Peripheral Interface — synchronous, full-duplex, master/slave bus	SPI
MOSI	Master Out Slave In — data driven by SPI master to slave device	SPI
MISO	Master In Slave Out — data driven by slave back to SPI master	SPI
SCK	Serial Clock — clock generated by the SPI master	SPI
CS / SS	Chip Select / Slave Select — active-low pin that enables one SPI device	SPI
CPOL	Clock Polarity — idle level of SCK (0 = idle-low, 1 = idle-high)	SPI
CPHA	Clock Phase — sample on leading (0) or trailing (1) SCK edge	SPI
I²C / TWI		
I ² C / TWI	Inter-Integrated Circuit / Two-Wire Interface — open-drain, multi-device 2-wire bus	I ² C / TWI
SDA	Serial Data line — bidirectional data; open-drain, needs 4.7 k Ω pull-up	I ² C / TWI
SCL	Serial Clock line — driven by master; open-drain, needs 4.7 k Ω pull-up	I ² C / TWI
ACK	Acknowledge — receiver pulls SDA low after each byte to confirm reception	I ² C / TWI
NACK	No Acknowledge — receiver releases SDA; used for last byte or on bus error	I ² C / TWI
TWBR	TWI Bit Rate Register — sets SCL: $f_{scl} = f_{cpu} / (16 + 2 \times TWBR)$	I ² C (AVR)
TWCR	TWI Control Register — START/STOP bits, TWINT clear, TWEN enable	I ² C (AVR)
TWDR	TWI Data Register — byte to transmit or the last byte received	I ² C (AVR)
TWSR	TWI Status Register — bus state status codes and prescaler selection	I ² C (AVR)
Other Communications		
USB CDC	USB Communications Device Class — MCU appears as a virtual COM port on PC	Communications
RS-485	Differential serial bus — multi-drop, up to 1200 m cable run, industrial	Communications
CAN	Controller Area Network — differential, priority-based; automotive and safety systems	Communications
NMEA	National Marine Electronics Association — GPS sentence format (\$GPGGA, \$GPRMC)	Communications
EMI	Electromagnetic Interference — noise from motors, VFDs; use differential comms to reject	Communications
Wireless & IoT		
LoRa	Long Range — chirp spread-spectrum radio (SX1276); low power, km-range uplink	Wireless / IoT

Abbrev.	Meaning	Architecture area
SF	Spreading Factor — LoRa: SF7 = fast, short range; SF12 = slow, long range	Wireless / IoT
ZigBee	IEEE 802.15.4 mesh radio — XBee module presents a UART interface to the MCU	Wireless / IoT
IoT	Internet of Things — networked embedded sensing/actuating devices	Wireless / IoT
SS	Smart Sensor — IoT edge node: senses, processes, and transmits	IoT Architecture
SA	Smart Actuator — IoT edge node: receives commands, controls physical process	IoT Architecture
LCR	Local Computing Resource — edge gateway, e.g. Raspberry Pi aggregating sensors	IoT Architecture
GC	Global Connector — internet gateway: 4G router, WiFi AP	IoT Architecture
RCR	Remote Computing Resource — cloud server: AWS IoT, Azure IoT Hub	IoT Architecture
Power & Hardware		
LDO	Low Drop-Out linear regulator — simple, low-noise; inefficient when $\Delta V > 2V$	Power / Hardware
GPIO	General Purpose Input/Output — MCU pin configured as digital input or output	Power / Hardware

Unit Relationships

Unit	Equivalent	Notes
Frequency / Time		
1 Hz	1 cycle/s	Timer period $T = 1/f$
1 kHz	1 000 Hz	ADC sample rates, PWM frequencies, PID loops
1 MHz	1 000 000 Hz	MCU clock speeds (16 MHz = 16×10^6 cycles/s)
1 ms	10^{-3} s	$f = 1$ kHz; typical PID / ADC period
1 μ s	10^{-6} s	$f = 1$ MHz; timer tick at 16 MHz, /1
1 ns	10^{-9} s	Single clock cycle at 1 GHz
Data rate — bits vs bytes		
1 bps	1 bit/s	Raw bit rate
1 kbps	1 000 bps	
1 Mbps	1 000 000 bps	Note: <i>lower-case</i> b = bit
1 B/s	8 bps	1 byte = 8 bits
1 KB/s	1 024 B/s	
1 MB/s	1 024 KB/s $\approx$ 8 Mbps	USB Full-Speed 12 Mbps $\approx$ 1.5 MB/s
UART	baud /10 B/s	10 bits per byte (start + 8 data + stop); 115 200 = 11,520 B/s
SPI @ 8 MHz	1 MB/s	8 bits/byte, back-to-back
I ² C @ 400 kHz	50 KB/s	8 data + 1 ACK bit per byte
Memory / Storage		
1 byte	8 bits	
1 KB	1 024 bytes	SRAM: 8 KB = 8,192 bytes
1 MB	1 024 KB	Flash: 128 KB; SD cards in GB range
Current / Power		
1 mA	10^{-3} A	Active MCU at 5 V, 16 MHz $\approx$ 10–20 mA
1 μ A	10^{-6} A	AVR power-down sleep $< 1 \mu$ A
1 mAh	1 mA $\times$ 1 h	Battery life (h) = $C_{\text{mAh}}/I_{\text{avg mA}}$
1 mW	10^{-3} W = $V \times I$	At 5 V, 20 mA: $P = 100$ mW
Voltage thresholds		
1 mV	10^{-3} V	ADC resolution (5 V, 10-bit) = 4.88 mV/step
V_{IH}	Logic-high input	TTL: > 2.0 V; 3.3 V CMOS: > 2.0 V; 5 V CMOS: > 3.5 V
V_{IL}	Logic-low input	TTL: < 0.8 V